An Empirical Framework for REST vs. gRPC Protocols, Isolating Protocol Metrics Across Multi-Paradigm Cloud (AWS) Database Architectures

Shlok Doshi (B039), Moksh Jain (B054), Aranck Jomraj (B057), Kahaan Kakabalia (B058)
Distributed Computing, Dr.Prashasti Kanikar
B.Tech, Sem VI
Mukesh Patel School of Technology Management and Engineering, NMIMS, Mumbai

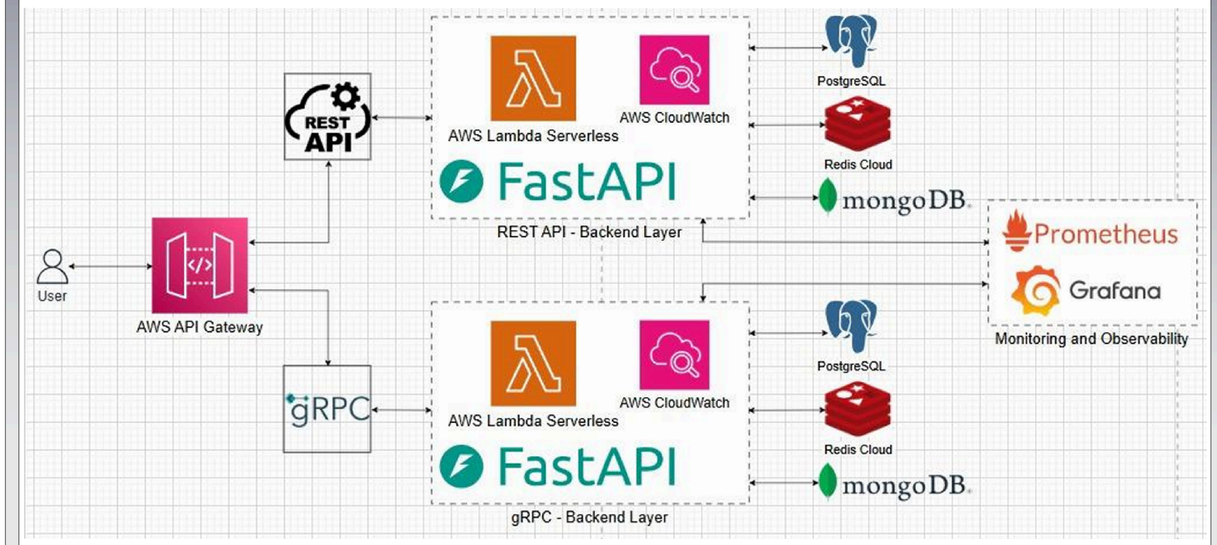
Problem Statement

gRPC is empirically fast than REST, and engineers cannot be confident that the speed difference will hold in an actual distributed production system. The current benchmarks do not take into account database bottlenecks, omit tail latency and do not consider the cost of migration itself. This results in teams basing their decisions on artificial outcomes which might not be applicable to their situation. By Amdhalds Law, a faster protocol has no effect when the bottleneck is the database. And even where migration is rational, no one has quantified the cost of running both stacks at the same time in the process of migration.

Limitations based on Literature Survey

- **No migration thresholds:** benchmarks indicate that gRPC is faster but no framework to specify when migration is justified.
- **Database blind spot:** none of the studies experiment on whether the database latency leads the request path and protocol speed.
- **Lack of tail latency results:** the literature only gives mean latency, not p99 comparison in cases of CPU and GC stress.
- **Unquantified migration cost:** no studies quantify dual-stack overhead, or schema versioning load in the live migration contexts
- **No local-to-cloud coefficient:** local benchmark measurements are considered to be universal and do not adjust to the cloud environment.

Proposed Architecture



Brief description of the components

- **AWS Lambda**: Serverless compute engine, stateless, and executable functions, which have infinitely scalable execution, and no infrastructure overhead.
- **FastAPI:** An asynchronous web app framework built on Python and non-blocking I/O to support thousands of concurrent request pipelines on a single thread event loop.
- **gRPC APIs:** Fast, binary RPC system with HTTP/2 multiplexing and Protocol Buffer serialisation to ensure blocking-free operation and to minimise serialisation costs.
- **REST APIs**: Standard paradigm of HTTP/1.1 that provides the performance base line and bears the ability to measure the performance of JSON serialisation in concurrent load.
- **PostgreSQL:** Commercial relational RDBMS that represents the heavy-Computing layer, stress-test efficiency of protocols under complicated ORM functions.
- **Redis:** In-memory data structure store that sub-milliseconds long, and removes database I/O as a latency variable, which measures the precise deviation in protocol behavior.
- **MongoDB:** Flexible, document-oriented, and NoSQL engine with schema flexibility to benchmark protocol behaviour with unstructured, variable-length payload retrieval at scale.
- **Prometheus:** Pull-based metrics collection engine scraping Four Golden Signals, with a one-second resolution, to allow tail latency (p95/p99) to be accurately measured.
- **Grafana:** Visualisation and interactive analytics platform on which a performance dashboard based on cross-protocol, cross-data metrics are overlaid, in real time.

Methodology

A FastAPI-based AWS Lambda serves as the backend to REST and gRPC, with Redis, MongoDB, and PostgreSQL. Prometheus monitors p99 latency and four golden signals with increasing load, where CPU and GC stress is injected to model production conditions. A Strangler Fig redirects a 5% traffic to gRPC with automatic rollback in case latency limits are violated.

Key benefits

- Determines the point at which gRPC migration will bring actual benefit and when it is consumed by database latency.
- First-ever controlled comparison of p99 of the REST and gRPC under real-stress.
- Measures a dual-stack migration overhead that is presently being estimated by teams.
- Provides practical thresholds that can be used by engineers opposed to interpolating based on synthetic benchmarks.

Conclusion

gRPC performed better in high concurrency situations, and the largest improvements were against Redis as protocol overhead was the only limiting factor, with PostgreSQL and MongoDB having decreasing returns due to ORM complexity and payload size. P99 tail monitoring through Prometheus revealed the behavior of degradation behavior that was not observable by metrics such as mean-based measurements, our dual-stack system measured real transition overhead, and AWS deployments showed that local benchmark measurements are not always applicable to high-performance cloud environments.